DGN-01 – Diagnóstico de aprendizajes previos

Bases de Datos NoSQL – Semana 1 · Unidad de Emprendimiento del SENA

Aprendiz: María Paula Herrera Rivas
Fecha: 15 de octubre de 2026
Entorno: Windows, Python 3.14.6, terminal de VS Code
Barrera de acceso o instalación observada: ninguna para el diagnóstico (solo se usa la biblioteca estándar de Python).

Parte A. Diagnóstico teórico

1. ¿Qué representan una tabla, una fila y una clave primaria?
Una tabla agrupa registros del mismo tipo, por ejemplo la tabla `iniciativas`. Cada fila es un registro completo, o sea una iniciativa con todos sus datos (código, nombre, sector, estado). La clave primaria es el campo que identifica cada fila de forma única y no se repite, por ejemplo `codigo = 'INI-001'`. Así no hay dos iniciativas con el mismo identificador.

2. ¿Para qué sirve una clave foránea? ¿Qué problema genera una iniciativa que referencia a un emprendedor inexistente?
La clave foránea relaciona una tabla con otra. En `iniciativas` podría haber un campo `emprendedor_id` que apunta al `id` de la tabla `emprendedores`. Si una iniciativa apunta a un emprendedor que no existe, queda un registro "huérfano": no se sabe quién es el responsable, las consultas con JOIN no la muestran y los reportes salen incompletos. La clave foránea evita eso porque la base de datos no deja guardar una referencia a algo que no existe.

3. ¿Qué devuelve `SELECT codigo FROM iniciativas WHERE sector = 'tecnologia';`? ¿Modifica registros?
Devuelve solo la columna `codigo` de las iniciativas cuyo sector es exactamente `'tecnologia'`. No modifica nada, porque `SELECT` solo consulta; para cambiar datos se usan `INSERT`, `UPDATE` o `DELETE`.

4. Diferencia entre una lista y un diccionario en Python. ¿Qué usarías para una iniciativa y para varias?
Una lista guarda elementos en orden y se accede por posición (`lista[0]`). Un diccionario guarda pares clave–valor y se accede por el nombre de la clave (`iniciativa["codigo"]`). Para **una** iniciativa usaría un diccionario, porque cada dato tiene su nombre. Para **varias** iniciativas usaría una lista de diccionarios, como en el ejercicio P2.

5. ¿Son equivalentes `false` y `"false"` en JSON?
No. `false` sin comillas es un booleano. `"false"` con comillas es una cadena de texto que tiene las letras f-a-l-s-e. Si un programa espera un booleano y le llega la cadena, puede tomar decisiones equivocadas; por ejemplo, en Python una cadena que no está vacía cuenta como verdadera aunque diga "false".

6. Diferencia entre campo ausente, campo con `null` y campo con cadena vacía.

- **Ausente:** la clave ni siquiera existe en el registro. Ej.: `{"codigo": "ASE-001"}` sin `fecha_realizacion`.

- **`null`:** la clave existe pero su valor es nulo a propósito. Ej.: `"fecha_realizacion": null` para decir que la asesoría todavía no se ha hecho.
- **Cadena vacía:** la clave existe y tiene un texto de longitud cero. Ej.: `"observacion": ""`.

Son cosas distintas y el sistema necesita una regla para cada una; no es buena idea usar `""` en lugar de una fecha desconocida.

7. ¿Qué ventaja tiene una función que retorna datos frente a otra que solo los imprime?
Si la función retorna, el resultado se puede guardar en una variable, usar en otra parte del programa, comparar en una prueba o mostrar de otra forma. Si solo imprime, el dato aparece en pantalla y ya, no se puede reutilizar ni probar automáticamente.

8. Si falla la lectura de un archivo JSON, ¿qué revisarías antes de cambiar el programa?
Primero el mensaje de error completo. Luego que la ruta y el nombre del archivo estén bien y que la terminal esté en la carpeta correcta, que el archivo no esté vacío, que la codificación sea UTF-8 y que el contenido sea JSON válido (comillas dobles, `true`/`false` en minúscula, sin comas al final). Muchas veces el problema está en el archivo o en la ruta, no en el código.

9. ¿Qué información reconoces en `2026-10-06T13:00:00Z`? ¿Qué ambigüedades evita frente a `06/10/26 8:00`?
Es el 6 de octubre de 2026 a las 13:00:00, y la `Z` indica que está en hora UTC. En Colombia (UTC−5) corresponde a las 8:00 a. m. del mismo día. Con `06/10/26 8:00` no se sabe si es 6 de octubre o 10 de junio, si el año es 2026 o 1926, ni en qué zona horaria está la hora. El formato ISO 8601 con zona evita todas esas dudas.

10. Se publicó por error una contraseña en Git. ¿Qué harías? ¿Basta con borrarla del archivo actual?
No basta, porque la contraseña queda guardada en el historial de commits y cualquiera con acceso al repositorio la puede ver. Haría esto:

1. Avisar de inmediato al responsable.
2. Cambiar (rotar) la contraseña, porque ya debe considerarse expuesta.
3. Quitarla del código y pasarla a un archivo `.env` incluido en el `.gitignore`.
4. Limpiar el historial con una herramienta como `git filter-repo` y revisar si alguien la usó.

Parte B. Diagnóstico práctico

Carpeta `diagnostico_nosql`, ejecutando cada archivo con `python nombre_archivo.py`.

P1. Corregir y leer JSON (`p1_json.py`)

Error inicial:

```
Error al cargar el texto original: Expecting value: line 1 column 31 (char 30)
```

Correcciones y por qué:
1. `True` → `true`. En JSON los booleanos van en minúscula; `True` con mayúscula es la forma de Python, por eso aparece “Expecting value” justo en esa posición.
2. Al corregir lo anterior apareció un segundo error: JSON no permite una coma después del

último elemento de una lista. Por eso cambié `["Validar mercado",]` por `["Validar mercado"]`.

La intención se mantiene: la iniciativa INI-001 está activa y su interés es “Validar mercado”. Dejé la versión original en el archivo dentro de un `try/except` para no ocultar el intento fallido.

Resultado:

```
codigo: INI-001 -> tipo: str
activa: True -> tipo: bool
```

Al cargar el JSON, `true` se convierte en `True` de Python, de tipo `bool`.

P2. Filtrar registros (p2_filtrar.py)

```
def seleccionar_pendientes(registros, sector):
    codigos = []
    for registro in registros:
        if registro.get("sector") == sector and registro.get("pendiente") is
True:
            codigos.append(registro["codigo"])
    return codigos
```

Campo ausente: uso `.get("pendiente")`, que devuelve `None` si la clave no existe, en vez de generar un error. Como `None` no es `True`, una iniciativa sin el campo **no** se selecciona: si no hay un dato que diga que está pendiente, no puedo asumirlo. Por eso INI-004 no sale aunque sea de tecnología.

Cadena vs. booleano: comparo con `is True` para aceptar solo el booleano. Si llegara `"pendiente": "True"` como texto, no se cuela. En Python una cadena con texto cuenta como verdadera, así que hasta `"False"` pasaría un `if` normal.

No modifico la lista: solo la recorro y armo una lista nueva.

Pruebas:

```
Pendientes de tecnologia: ['INI-001']          ← prueba con resultados
Pendientes de economia_circular: []            ← prueba sin resultados
Con pendiente como texto: []                   ← prueba extra con cadena
Todas las pruebas pasaron
```

P3. Consultar relaciones con SQL (p3_sql.py)

Consulta:

```
SELECT i.id, e.alias
FROM iniciativas AS i
INNER JOIN emprendedores AS e ON i.emprendedor_id = e.id
WHERE i.estado = 'activa'
ORDER BY i.id;
```

Cómo relacioné las tablas: con un `INNER JOIN` que une cada iniciativa con el emprendedor cuyo `id` coincide con su `emprendedor_id`. Después filtro solo las activas y ordeno por el identificador de la iniciativa.

Resultado:

```
[(101, 'Emprendedor A'), (103, 'Emprendedor A')]
```

Las dos iniciativas activas pertenecen al Emprendedor A. La 102 no aparece porque está archivada, y el Emprendedor C no aparece porque no tiene iniciativas.

Autoevaluación

- **Puedo realizar…** consultas `SELECT` con `WHERE`, recorrer listas de diccionarios en Python y escribir funciones que retornan datos, porque lo he trabajado en FastAPI y en proyectos con MySQL.
- **Necesito reforzar…** la sintaxis exacta de JSON (las diferencias con Python me confunden), la diferencia entre campo ausente, `null` y cadena vacía, y los `JOIN`, que no uso tan seguido.
- **Mi primera acción de mejora será…** practicar `json.loads` / `json.dumps` con varios ejemplos de errores, y repasar los JOIN con las tablas de este ejercicio antes de la semana 2.

Registro diagnóstico (para el instructor)

Componente	Máximo	Resultado	Fortaleza / necesidad de apoyo
Teoría	20		
P1. JSON	10		
P2. Python	10		
P3. SQL	10		
Total	50		